

JavaFX常用控件之



Group、StackPane和ScrollPane

北京理工大学计算机学院
金旭亮

Group



Group概述



Group是一个图形的容器，它可以用于组合多个Shape为一个整体，然后一次性地引用、显示或移动它们。它会自动计算这些控件的大小并且自动地扩展其边界。



Group内部使用ObservableList来管理子控件。因此，只要向其中加入或移除控件，Group就会自动重绘。这种绘制是按照控件加入的顺序，依次绘制每个控件的。



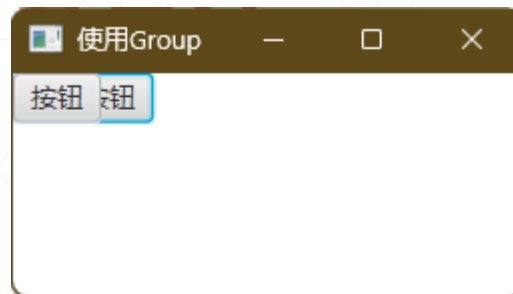
注意：Group并不负责排列控件，控件的显示位置和大小必须由程序员指定（可以使用setX、setY之类方法进行设定），设置不当的话，这些控件可能会彼此重叠。



施加于Group上的动画、变换、特效等操作，会自动地应用于所有的控件。

Group的基本使用方法

```
Button button1 = new Button( text: "一个按钮");  
Button button2 = new Button( text: "按钮");  
Group group = new Group();  
group.getChildren().add(button1);  
group.getChildren().add(button2);
```



默认情况下，Group会在左上角（0，0）处显示控件，因此，上述两个按钮将会重叠。



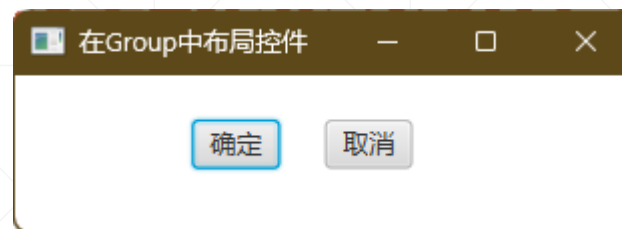
由此可知，如果使用Group显示多个控件，必须手动编写代码指定这些控件的显示位置以避免重叠。



Group本身尺寸改变时，不会重新排列它所包容的Shape或UI控件，如果想做到这点，必须使用功能更强的布局控件。

指定Group中控件的显示位置

```
Button okBtn = new Button( text: "确定");  
Button cancelBtn = new Button( text: "取消");  
  
// 设定两个按钮的显示位置  
okBtn.setLayoutX(80);  
okBtn.setLayoutY(20);  
cancelBtn.setLayoutX(140);  
cancelBtn.setLayoutY(20);  
  
Group root = new Group();  
root.getChildren().addAll(okBtn, cancelBtn);
```



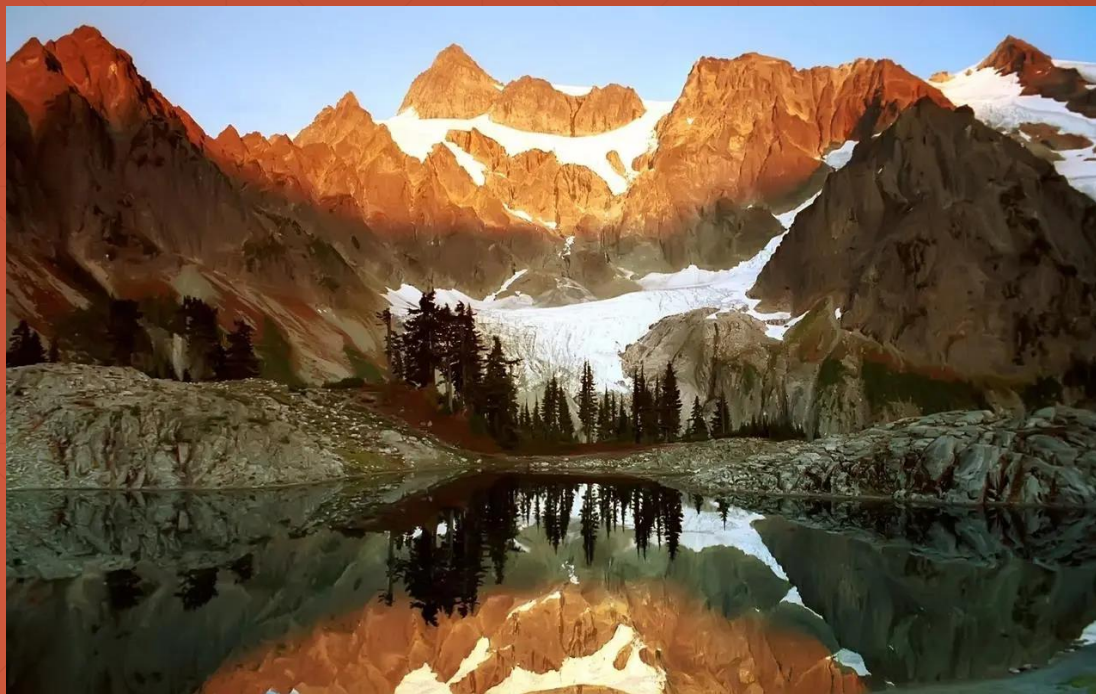
可以调用控件的`setLayoutX()`、`setLayoutY()`两个方法设定控件在Group中的显示位置。

“批量”应用特效与变换

```
//创建两个按钮
Button okBtn = new Button( text: "确定");
Button cancelBtn = new Button( text: "取消");
// 设定显示布局
okBtn.setLayoutX(70);
okBtn.setLayoutY(50);
cancelBtn.setLayoutX(120);
cancelBtn.setLayoutY(50);
//追加到Group控件中
Group root = new Group();
root.getChildren().addAll(okBtn, cancelBtn);
//给Group中的所有控件, 添加阴影、旋转变换, 并设置其透明度
root.setEffect(new DropShadow());
root.setRotate(10);
root.setOpacity(0.80);
```



针对Group设置的变换和特效，自动应用于全体。



StackPane

StackPane概述



StackPane派生自Pane，主要是给它添加了控件定位功能，可以设置控件的对齐方式，并且可以设置Margin。

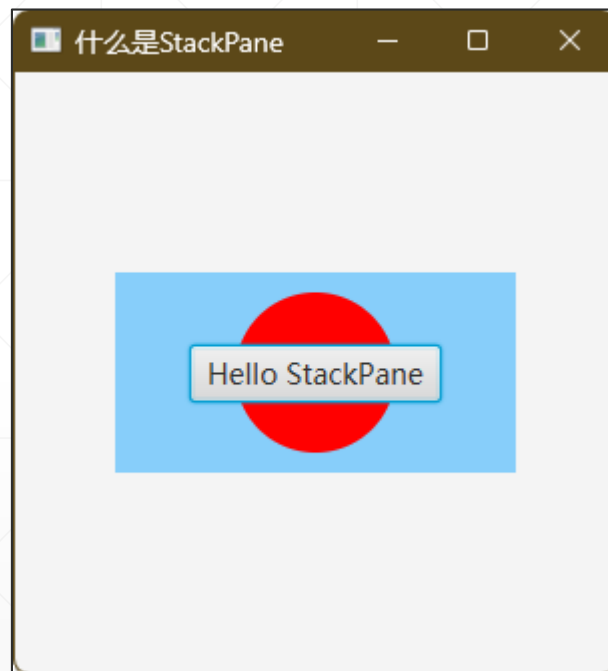
StackPane将一个控件一个控件地堆叠起来，后来者居上。

StackPane会尝试将所有控件调整为占满整个区域，如果某些控件尺寸受限，就把它居中显示。

使用StackPane

```
@Override
public void start(Stage stage) {
    //构建StackPane, 其参数的顺序决定了控件显示的顺序
    //后来者居上
    StackPane pane = new StackPane(
        new Rectangle( width: 200, height: 100, Color.LIGHTSKYBLUE),
        new Circle( radius: 40, Color.RED),
        new Button( text: "Hello StackPane")
    );

    stage.setScene(new Scene(pane, width: 300, height: 300));
    stage.setTitle("什么是StackPane");
    stage.show();
}
```



与样式相配合



对样式的支持，其实是从Region基类继承而来的。

```
Rectangle rect = new Rectangle(width: 200, height: 50);  
rect.setStyle("-fx-fill: lavender;" +  
              "-fx-stroke-type: inside;" +  
              "-fx-stroke-dash-array: 5 5;" +  
              "-fx-stroke-width: 1;" +  
              "-fx-stroke: black;" +  
              "-fx-stroke-radius: 5;");
```

```
Text text = new Text("A Rectangle");
```

```
StackPane root = new StackPane();  
root.getChildren().add(rect);  
root.getChildren().add(text);
```

```
root.setStyle("-fx-padding: 10;" +  
              "-fx-border-style: solid inside;" +  
              "-fx-border-width: 2;" +  
              "-fx-border-insets: 5;" +  
              "-fx-border-radius: 5;" +  
              "-fx-border-color: blue;");
```

设置控件对齐方式



默认情况下，StackPane自动将控件堆叠在中心。可以通过setAlignment()方法来更改特定控件的对齐特性

```
stackPane.setAlignment(lblInfo, Pos.BOTTOM_CENTER);
```



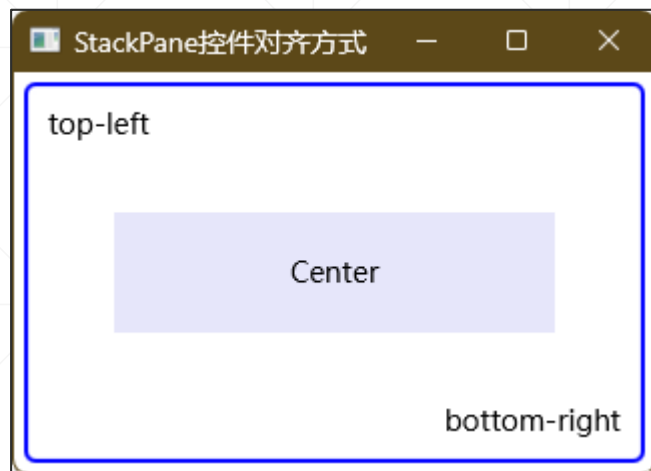
上述代码将标签控件显示在顶部中央。



如果不指定控件名，则会影响到所有的控件对齐方式：

```
stackPane.setAlignment(Pos.BOTTOM_CENTER);
```

对齐方式示例



```
Rectangle rect = new Rectangle( width: 220, height: 60);
rect.setFill(Color.LAVENDER);
//默认对齐方式是居中
Text center = new Text("Center");
Text topLeft = new Text("top-left");
StackPane.setAlignment(topLeft, Pos.TOP_LEFT);
Text bottomRight = new Text("bottom-right");
StackPane.setAlignment(bottomRight, Pos.BOTTOM_RIGHT);
//将所有控件追加到StackPane中
StackPane root = new StackPane(rect, center, topLeft, bottomRight);
root.setStyle("-fx-padding: 10;" +
              "-fx-border-style: solid inside;" +
              "-fx-border-width: 2;" +
              "-fx-border-insets: 5;" +
              "-fx-border-radius: 5;" +
              "-fx-border-color: blue;");
```




ScrollPane

ScrollPane概述

ScrollPane用于显示一个比较大的内容，内置水平和垂直两个滚动条以实现滚动查看。



示例先使用ImageView装载一幅图片，之后，把它加到ScrollPane中。

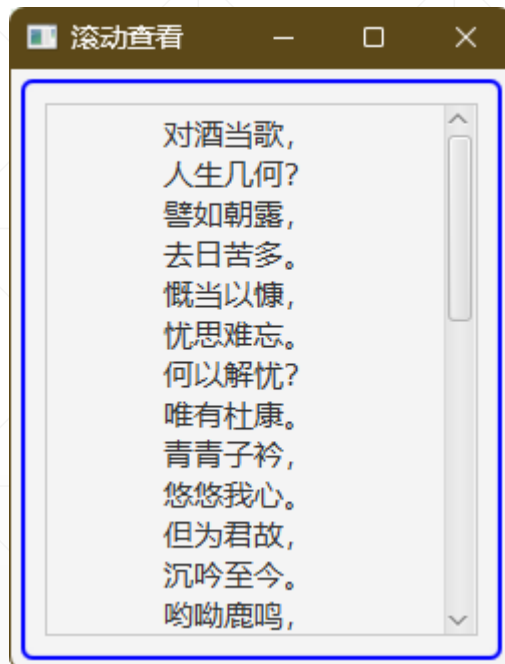
窗体中的顶层控件是VBox，它的大小比图片要小，将ScrollPane加入之后，滚动条出现，就可以滚动查看大图片了。

示例分析

```
@Override
public void start(Stage primaryStage) throws Exception {
    VBox vBox = new VBox();
    vBox.setPadding(new Insets( topRightBottomLeft: 10));
    //从资源文件夹中读取图片
    String image = getClass().getResource( name: "image.jpg").toExternalForm();
    Image grassland = new Image(image);
    ImageView imageView = new ImageView(grassland);
    //将ImageView加入ScrollPane, 再加入VBox
    ScrollPane scrollPane = new ScrollPane(imageView);
    vBox.getChildren().add(scrollPane);
    //480*320的场景大小, 明显小于图片, 所以, 滚动条是必然出现的
    Scene scene = new Scene(vBox, width: 480, height: 320);
    primaryStage.setScene(scene);
    primaryStage.setTitle("滚动查看大图片");
    primaryStage.show();
}
```

默认情况下, 使用滚动条来滚动查看内容。需要的话, 可以调用setPannable(true)将ScrollPane置为pan模式, 用鼠标直接拖动查看。

显示长的古诗



```
VBox root = new VBox();
root.setStyle("-fx-padding: 10;" +
    "-fx-border-style: solid inside;" +
    "-fx-border-width: 2;" +
    "-fx-border-insets: 5;" +
    "-fx-border-radius: 5;" +
    "-fx-border-color: blue;");

//从资源文件夹中读取一首古诗
var url = getClass().getResource(name: "poem.txt");
var poem = Files.readString(Path.of(url.toURI()));

//使用标签控件显示古诗内容
Label lblPoem = new Label(poem);
lblPoem.setFont(new Font(size: 15));

//居中显示
lblPoem.setAlignment(Pos.TOP_CENTER);

//利用数据绑定实现动态设置标签控件的宽度
lblPoem.prefWidthProperty().bind(root.widthProperty().subtract(other: 60));

//将标签控件放到ScrollPane中, 以便支持滚动查看
ScrollPane sPane = new ScrollPane(lblPoem);
sPane.setPannable(true);

//将ScrollPane放到VBox中
root.getChildren().add(sPane);
```